

MRK Hotels — Receptionist & Waiter Workflow: Problems & Solutions

Source documents: `RECEPTIONIST WORK FLOW(1).pdf` and `WAITER WORK FLOW.pdf`. Each problem is quoted from the PDF, followed by its solution and an honest status tag:

- **DONE** ✓ — fixed end-to-end (backend gate + frontend) in this project state.
- **IMPLEMENTATION** — fix agreed, still needs coding.
- **CONFIRM** — needs a product owner decision before work (behavioural/business choice).

1. RECEPTIONIST WORK FLOW

1.1 Non checked-in reservation

Problem 1.1.1 — Editing reservation is not live

USER can EDIT reservation while yet not checking in customer. Can edit guest details or booking details to change check in or check out. **CURRENTLY the changes are not live** meaning they are not reflective when changes are saved the TAB remains unchanged.

Solution (**DONE**): edits are live. The stay-dashboard Amend Stay / Room Move works for pending, confirmed and in-house stays and every save runs through `runStayAction`, which reloads the folio and — critically — re-points the modal at the freshly fetched row instead of staying bound to the stale bar (`HotelDashboard.vue` : after `load(true)` it re-selects the row by `reservation_id` and rebinds `activeBar`). The backend `update` accepts pending/confirmed/checked_in edits, so a reserved guest's details or dates can be corrected before check-in. Covered end-to-end by `e2e/hotel-folio-workflow.spec.js` A1 "amend stay edits are reflected live" (reopening the modal reads back the edited date) and backend `ReservationInHouseEditTest`.

Problem 1.1.2 — Pre-check-in payment wrongly turns the row green

USER can ADD payment of a customer that is not yet checked in but only reserved ... **CURRENTLY when the USER ADDS payment the color changes from RED TO GREEN as if the customer has been checked in while not. UNLESS the customer is CHECKED IN the color should not change.**

Solution (**DONE**): the row colour is bound to check-in state, never to payment. The stay tape colours from `status` (`toPanelBar` : `checked_in` → green, else red/blue per lifecycle); a reserved guest that has paid in full stays RED — only an actual check-in turns the bar green (`HotelDashboard.vue`, comment "A reserved guest that has paid in full must NOT flip green"). The reservation list badge likewise derives from `statusBadge(r.status)`, so adding a deposit never recolours the row; it only appears as a folio payment.

Problem 1.1.3 — Cancel booking vs. VOID after check-in

USER can CANCEL BOOKING of a reserved guest and data is completely erased but when CHECKED IN only management has access to VOID the reservation.

Solution (**DONE** ✓): VOID is now a management-level action. The API returns **403** for non-privileged roles on VOID, and the operator pads no longer render the VOID control (VOID lives on the cashier/management panel). Cancel of a *not-checked-in* booking remains available and fully clears the reservation as designed.

Problem 1.1.4 — Future-date reservations can be checked in early

USER should not be able to CHECK IN a reservation of a future date that is not CURRENT DATE. CURRENTLY the USER can ACTIVATE CHECK IN of a customer who reserved a room for 26/09/2026 which is a date that has not even reached yet ... when clicking check in it accepts.

Solution (DONE ✓): check-in is rejected when the reservation's check-in date is not the property's business *today*. `ReservationService::checkIn` aborts with **422** ("Cannot check in: the reservation check-in date (26/09/2026) is still in the future.") whenever the scheduled arrival is strictly after today — so a future booking cannot be activated early. The frontend surfaces the server message. Covered by `CheckInGateTest::test_check_in_is_rejected_when_the_check_in_date_is_still_in_the_future`.

Problem 1.1.5 — Move room (reserved, not checked in)

USER can MOVE ROOM of reserved (NOT CHECKED IN) guest ... perfectly this works perfectly.

Solution (DONE ✓): works as intended — no change required.

Problem 1.1.6 — Folio should itemise each night; labels are generic

Folio Operation when guest is checked in should consists of all nights based on number of nights; also currently DESCRIPTIONS reads as RENTAL CHARGES.

Solution (DONE ✓): when checked in, the folio lists one line per night for the stay. Each line's description is **date-aware and names the room** (Room 101 · Rent 21/09/2026), never the generic RENTAL CHARGES catch-all — `NightAuditService::roomChargeDescription` stamps an individual row for every night at check-in and when the stay is edited, so each night is identifiable for audit/due-out. Covered by `ReservationInHouseEditTest` (long-stay items each carry their own dated line).

1.2 Questions when guest is checked in

1.2.1 — SMS no longer appears when payments are made / on reservation setup

Solution (DONE ✓): SMS is event-driven and asynchronous and is connected at both points the PDF wants. `EVENT_BOOKING_CONFIRMED` fires on reservation setup and `EVENT_CHECKOUT` on departure (`ReservationService.php`), and payment receipts send from `PaymentService.php` — each via `SmsService::sendForEvent`, so a guest SMS goes out on booking/check-in and again after a payment. A failed send surfaces as a visible deliverable status instead of failing silently.

1.2.2 — Can a user EDIT a reservation while the guest is CHECKED IN?

Solution (DONE ✓): money-affecting edits for an in-house guest go through **Amend Stay** and reconcile to the folio live. Moving rooms re-prices every posted night at the new room's rate (and re-names each line to the current room), re-deriving `total_amount`, the running `ROOM CHARGES` (past-night delta only) and `BALANCE` — covered by `test_in_house_room_move_reprises_every_night_on_the_folio`. Extending/shortening the stay posts or releases the trimmed nights (past nights already closed by a completed business day stay reflected). Free-text/profile edits remain editable without touching the folio.

1.2.3 — Amend stay / edit check-out must reflect live on room charges & folio

...do TOTAL ROOM CHARGES and ROOM CHARGES in folio operations or BALANCE reflect LIVE changes?

Solution (DONE ✓): after any room-charge/date edit the server recomputes the stay bill and the folio aggregates from the same item set and the frontend reloads the folio. `ReservationService::repriceRoomChargeRows` keeps the frozen stay bill (`total_amount` = the sum of the per-night rows) equal to the ledger, and the room-change branch in

`ReservationController::update` re-runs `reconcileStayBill` so `TOTAL ROOM CHARGES / ROOM CHARGES / BALANCE` never drift from each other after a move or date edit.

1.2.4 — Editing the CHECK IN date once the guest is checked in

User should not be able to EDIT CHECK IN of the guest once is checked in either to previous date (which should be impossible once night audit is done) or to future date.

Solution (DONE): the check-in date is immutable once the guest is in-house — the API rejects any edit that pushes arrival to an earlier or later date (`ReservationController::update` → "The check-in date of an in-house guest cannot be changed.") and the stay ledger is recomputed from the booked nights. Covered by `test_checked_in_check_in_date_is_immutable`.

1.2.5 — Add payment to a checked-in guest

Solution (DONE ✓): supported; payment appears as a positive folio line and reduces the balance. Folio Operation reflects the new balance immediately.

1.2.6 — DUE OUT logic & the missing purple colour

IS DUE OUT feasible? ... the NIGHT AUDIT of 21/09/2026 may not be done until guest has paid and CHECKED OUT. WHY DOES DUE OUT NOT HAVE ITS PURPLE COLOR?

Solution (DONE ✓): a guest whose stay end has been passed by the night audit becomes **Due Out** and is flagged purple. The colour is computed from `check_out >= night-audit date` (`toPanelBar` tints `bar-purple` when the departure date has been reached) and rendered on the panel (`HotelDashboard.vue`), so a due-out guest reads as due out at a glance.

1.2.7 — Folio balance posting to creditors

The user tried to post an outstanding balance ... and gave ... "This folio has no outstanding balance to post."

Solution (DONE ✓): outstanding balance for creditor posting is computed as `folio charges - payments - voided - transfer-out`. The two failure cases are now told apart in `FolioController::postToCreditors`: - no outstanding balance → "This folio has no outstanding balance to post.", - a company mapped to the reservation but **no postable creditor account configured** → 422 "No creditor account is configured for this company...", so "nothing to post" and "no creditor configured" are never confused.

1.2.8 — Sending invoice by email reads as a server error

Why does sending invoice through email reads as server error?

Solution (DONE ✓): the mail facade import was wrong, so every send fell over. Fixed (`InvoiceController` now uses `Illuminate\Support\Facades\Mail`); invoice emails now send and any real transport failure surfaces as a clean message instead of a 500.

1.2.9 — Discount / negative adjustment increases TOTAL PAID

Why does APPLYING DISCOUNT & USING NEGATIVE ADJUSTMENT increases TOTAL PAID? TOTAL PAID should remain the same when these features are used only BALANCE should change.

Solution (DONE ✓): TOTAL PAID stays unchanged for discounts and negative adjustments; only BALANCE moves. TOTAL PAID is derived strictly from settled cash/card/folio settlements — the guest portal's `totalPayments` (and the folio summary it drives) sums only payments with a settled status (`completed / paid / confirmed`), never charge adjustments or pending/failed attempts (`GuestPortalController::folio`).

1.2.10 — Does ADD INCLUSION work / appear in folio operations?

Solution (DONE ✓): ADD INCLUSION creates a folio line and shows in Folio Operation, and the folio is refreshed after the addition so the line (and the balance it lifts) is visible on the invoice printout. No further change required.

1.3 Transfer folio & splitting folio

Can guest folio balance be settled ... transfer some FOLIO particulars from one guest to another ... e.g. a guest has requested 3 rooms and wants all bills concentrated to one room.

1.3.1 — Viewing one folio zeroes the other

there is a folio TAB at the TOP ... WHY WHEN USER CLICKS TO VIEW ONE FOLIO the other one's balance converts to 0 ... why do both sometimes looks clicked while only one is selected to VIEW?

Solution (DONE): balances are now computed per-row and never nulled by a view switch. The current-folio row derives its figure from the loaded stay folio (`folioRowBalance` , `HotelDashboard.vue`) — it no longer falls back to a missing `balance_due` on the bar object that rendered blank TZS 0.00. Only the one selected folio ever carries the active row state, so the switch table shows the right balance on every row at all times. (Selection-state is single-sourced from `activeFolioId` .)

1.3.2 — Add payment on the RELATED folio turns the CURRENT balance negative

When user tried to ADD PAYMENT of a RELATED FOLIO on CURRENT FOLIO TAB the BALANCE turned to NEGATIVE on CURRENT FOLIO BALANCE AND BALANCE DUE.

Solution (DONE): payments are now applied to the **viewed folio** — `submitPayment` posts to `activeFolioId` (the folio on screen), not the hard-coded "current" bar. And per the PDF's own ask, a **transfer dissolves the "RELATED FOLIO"**
linkage: `related_folios` is now built only from `split_*` / `cut_*` provenance rows in both `ReservationController::folio()` and `FolioController::relatedFoliosPayload()` , so a `transfer_out` / `transfer_in` pair no longer surfaces a switchable chip. One bill owns one balance.

1.3.3 — Charges/inclusion/discount only touch the CURRENT folio

When ADDING CHARGES|INCLUSION|APPLYING DISCOUNT after splitting bills (CURRENT FOLIO and RELATED FOLIO) why do all changes only appear on the CURRENT FOLIO even when the user is VIEWING RELATED FOLIO?

Solution (DONE): charges and room-charge postings already write to `activeFolioId` (the viewed folio); payments now match. The viewed/active folio id is plumbed into the folio operations so every charge/inclusion/discount lands on the folio the user is actually looking at.

1.3.4 — PRINT INVOICE auto-downloads instead of showing the print page

When user CLICKS PRINT INVOICE why does it automatically download the invoice instead of redirecting to PRINT PAGE?

Solution (DONE): the stay-view **Print invoice** action no longer downloads a blob — it opens the professional print document (folio header, room charges / total paid / refund / net balance table, "printed by" footer) in a new window that auto-fires the print dialog, the same clean sheet the ledger reports use (`HotelDashboard.vue` `printInvoiceBreakdown`). The labelled **Download/Invoice** buttons in the reservation detail and payment list still save the backend PDF — that is intentional and separate from printing.

1.3.5 — Invoice has no margins/layout

Why does INVOICE not have MARGINS and LAYOUT?

Solution (DONE): the invoice/folio print document carries the full print stylesheet — `@page A4 portrait; margin: 12mm`, centered hotel name, title, meta row, bordered right-aligned number table, and footer — so it renders as a properly paginated, margined A4 print like every other receivable printout.

1.3.6 — New folio for a current guest changes the ROOM NUMBER in the FOLIO NAME CODE

Why does creating new folio for current guest changes room number ... the name of the guest remains the same but the room number changes?

Solution (DONE): the FOLIO NAME CODE keeps the guest's assigned room. Opening an additional folio for a current guest (`POST .../folio/open-new-folio`) inherits the stay's `room_id` — the frontend sends no room, and `createSplitFolio` copies the source room — so the new folio reads the same `guest_name · room` as the original. Locked by `test_open_new_folio_keeps_the_guests_assigned_room`.

1.3.7 — After a split, can 2 independent invoices be printed / sent?

If bill is split can user print 2 independent invoices since there are 2 folios? ... can user send 2 independent invoices since there are 2 folios?

Solution (DONE): yes — after a split/cut the folio switcher lists each folio row with its own **print** (fetches that folio's payload and opens its invoice breakdown) and **send** actions (`HotelDashboard.vue` `openInvoicePreview` / `sendFolioInvoice` on the row's `reservation_id`), so the receptionist prints or e-mails 2 independent invoices, each carrying its own folio code, balance and invoice.

1.4 Upon check out

1.4.1 — Void reservation = management only

Void Reservation access should only remain to management.

Solution (DONE ✓): VOID (and closed-order VOID) is gated to management/privileged roles; the API returns **403** for waiters and operators, and the VOID control is hidden on their pads.

1.4.2 — Edit folio operations after checkout = management only

Access to edit folio operations after guest checking out should only remain to management.

Solution (DONE ✓): after check-out, editing and voiding folio entries is blocked for operators — `FolioController:update` and `void` bail with **403** unless the caller is a manager/accountant/ owner/superadmin once the reservation is `checked_out` (`assertCanManageFolioEntries`). Management may reopen/edit the closed folio, and any change remains reflective even after day-close.

2. WAITER WORK FLOW

2.1 Waiter's major functionalities

Waiter's major functionalities: to place an order; to print bill.

2.1.1 — Remove the WELCOME "USERNAME" + CONTINUE tab

there is a little TAB in the middle of the screen that says WELCOME "USERNAME" and a button to CONTINUE, if a user is going to visit the system more than 10 times a day this looks like a time waster please remove it.

Solution (DONE ✓): the welcome/continue interstitial is removed — login lands straight on the pad (single-role/single-terminal logins). No per-session SweetAlert; a landing page is only ever gated to multi-role users.

2.1.2 — Can the waiter place / see orders?

Solution (DONE ✓): waiters place new orders and see their own open orders in the "Open orders" list, including links back to the ticket.

2.2 New orders — table handling

2.2.1 — Selecting a table marks it occupied

When waiter select table does the table marks as occupied?

Solution (DONE ✓): selecting a table marks it occupied immediately, from the first order item.

2.2.2 — Other waiters see a taken table

...do other waiters when they login see if the table is taken?

Solution (DONE ✓): occupancy is now keyed by **user id** (`created_by`), not display name, and is visible to every waiter on login, so a taken table is shown to all staff in real time. (This also fixes name-collision lookups.)

2.2.3 — Waiter cannot add to a table she/he already occupies

Can waiter add an order to an occupied table by her? Currently the waiter cannot ... it says "THAT TABLE IS OCCUPIED. PICK A FREE TABLE".

Solution (DONE ✓): a waiter can reopen a table *that she/he occupies*. Occupancy is keyed by user id, so "same waiter → reopen (loads their existing ticket to continue); different waiter/role → blocked; manager always opens". The header **SearchableSelect dropdown was wired through** `selectTable()` too (`onTableDropdownChange`), so picking the waiter's own occupied table by search continues its ticket instead of silently starting a fresh order — the same rule as tapping the table map tile.

2.2.4 — Item-tab resets when menu items are chosen before the table

Why when user starts by selecting MENU ITEMS first then follows by selecting A TABLE does the TAB that receives the selected menu items RESETS itself as from the start to reselect the menu items again?

Solution (`DONE` ✓): selecting a table no longer clears the working item tab. `selectTable()` preserves the picked items when the user goes menu-first (items chosen, then a free table); only stepping away from a *resumed* ticket (a continued live order) starts a clean slate — that would be wrong to carry into a brand-new table.

2.3 Open orders

2.3.1 — Show order date & time, running time, and the waiter's name

Do open orders show date & time when the order was taken? Do open orders show how much time the order has been running? Do open orders show the name of the waiter that placed the order?

Solution (`DONE`): open-order cards display **taken-at date/time** (clock row with `created_at / order_date`), a live **elapsed running timer** (stopwatch row, refreshed every 30s), and the **waiter name** (from the order's `waiter_name`, backed by `created_by`), so F&B can see which orders are ageing and who holds them.

2.3.2 — Split / transfer of an open order

Can an open order be split? Can an open order be transferred?

Solution (`DONE` ✓): split is a **cashier/supervisor** action; a merged open order can be split back into its table folios. Transfer moves the order (or selected items) to another table — backend `OrderController::transferOrder / transferOrderItems / splitOrder` — and, per the PDF, the merged order **appears on the side the orders were transferred TO** (target table/folio).

2.3.3 — Reprint KOT must be watermarked

Can waiter reprint KOT of an open order? If a KOT is reprinted should be written at the top that the order is REPRINTED.

Solution (`DONE`): KOT reprints carry a ***** REPRINTED ***** watermark stamped between the header and the ticket number, so a reprint can never be mistaken for the original. Applied to every KOT printed from the waiter pad's open-order board and the cashier's reprint action (`utils/receipts.js` `kitchenTicketLines` + `opts.reprinted`, tested in `src/_tests_/receipts.spec.js`).

2.3.4 — Merge menu items from 2 tables into ONE printable bill

Can waiter merge menu items from 2 different tables to create ONE BILL that is printable? Currently menu items can only be placed on the same table BUT they are not on the same TAB therefore how can the total bill that consists of food and drinks be printed?

Solution (`DONE` ✓): waiters and cashiers merge items across tables into one printable bill when they share a guest/folio context. `OrderController::mergeOrder` moves every item of the source order onto the target table's open order (creating one if none) and closes the emptied source; the printer combines the selected orders even though the items arrived on different service tabs, producing a single combined ticket. Also used by the bar+restaurant table-transfer merge (see 2.3.5).

2.3.5 — Bar + restaurant merged order: which side does it appear on?

If a bar order and restaurant order are merged through table transfer which side will the merged order appear? The merged order should appear to the side to which the selected orders were transferred to.

Solution (`DONE`): the merged order takes the **destination** of the transfer — `mergeOrder` moves all items onto the target table's open order (creating one if none) via `target_table_number`, and the emptied source order is closed (`cancelled`). The merged bill appears on the side the orders were transferred TO; the originals are removed from the source side.

2.3.6 — REMOVE status-advance from the waiter panel (RUNNING/SETTLED only)

Why do waiters panel open orders have access to set ORDER IS READY or ORDER IS SERVED? There should be only two conditions RUNNING or SETTLED ... REMOVE "START PREPARING" / "THE BELL SIGN" (order is ready) / "THE DISH SIGN" (order is served).

Solution (DONE ✓): waiters no longer drive the service flow. - **Backend**: the API returns **403** when a waiter calls any status advance (preparing / ready / served at order or per-item level) or VOID — the kitchen and bar lead the flow. - **Frontend**: the waiter pad no longer renders **START PREPARING / ready (bell) / served (dish)** or the **VOID** control (isWaiterRole guard + waiterCannotAdvance message). - The waiter keeps exactly two lifecycle viewpoints: **RUNNING** (open) and **SETTLED** (closed).

2.4 Closed orders

2.4.1 — View the menu items of a closed order

Can waiter see the menu items of a closed order? Waiter should be able to view closed order consists of which menu items.

Solution (DONE ✓): the **Order Summary** tab lists every one of the waiter's tickets for the day (including settled/closed ones; the backend eager-loads items). Each row now has a **View items** toggle that expands the item lines (qty × name + accompaniment + line amount). A ticket that arrives without an item payload falls back to GET /orders/{id} to fetch them, so "what was on that ticket" is always answerable.

2.4.2 — Does the table free when the order is settled?

When an order is settled does the table become free?

Solution (DONE ✓): the table is released exactly when the last open ticket on it is settled — OrderController::pay() flips the order to **completed** then calls freeTableIfCleared() , which only frees when no remaining order on the table is outside ['completed', 'cancelled'] . The void and bill-to-room paths trigger the same release. Covered by test_table_auto_occupies_on_order_and_frees_when_settled (auto-occupies on order, stays occupied while a second ticket runs, frees on final settle, frees on void, reserves/out-of-service never auto-flipped).

2.4.3 — Reprint a closed order's KOT (labelled as closed)

Can waiter reprint KOT of a closed order? Waiter should be able to reprint closed order but should be able to indicate that it's a closed order.

Solution (DONE ✓): a closed order (completed/cancelled) in the **Order Summary** tab carries a **Reprint KOT** action. The reprint goes through the shared reprint path with a **CLOSED ORDER — REPRINT** watermark stamped under the ticket header (distinct from the plain REPRINTED stamp used for open tickets), so the kitchen never mistakes it for live work. The cashier's summary reprint uses the same closed watermark. Covered by a receipts unit test for the closed watermark placement.

2.4.4 — VOID of a closed order is management-only

Can a closed order be voided? THE MANAGEMENT should be able to VOID closed order but the VOID option should not appear on waiters panel but cashier's. VOIDING a closed order will affect the reports and changes should be reflective even after day close is done.

Solution (DONE ✓): VOID of closed orders is **management/cashier only**. The waiter panel has no VOID control, and the API returns **403** for a waiter VOID attempt. Voided amounts flow through to the reports and remain reflective even after day-close.

2.5 Dashboard

2.5.1 — LOW STOCK tab is not reflective

Is LOW STOCK tab that shows stock at hand and reorder level REFLECTIVE? Currently it is not reflective since the user issued and accepted indent of several items such as AZAM JUICE but they all still read 0.

Solution (**DONE** ✓): stock-at-hand is recomputed from the department shelf balances (**purchase receipts + accepted indents (goods-in) – issues/consumption**), not the raw stock column — `InventoryService::adjustStock` applies every receipt/indent/adjustment to the shelf pivot and `InventoryItem::stockFor` reads that balance, so an item issued and accepted no longer reads 0. The list refreshes **live**: a new `InventoryUpdated` broadcast event is fired on every stock movement and the staff dashboard (`OrderTakerDashboard`) and store dashboard reload their LOW STOCK / stock snapshot the moment a GRN lands or an indent is supplied/accepted (`useStockRealtime` on the tenant channel).

2.5.2 — FAST MOVING ITEMS should show live changes

Is FAST MOVING ITEMS tab reflective and shows live changes?

Solution (**DONE** ✓): ranked by live ticket/sales volume and refreshed on every F&B order change. The bar/cashier staff dashboard listens to the `.order.updated` tenant push and reloads the FAST MOVING list instantly as new tickets/items land (`ReportController::staffDashboard` ranks the 8 most sold items of the active day/department, `loadDashboard` on the dashboard tab), so the list updates in real time instead of on a stale snapshot.

Status summary

Area	Items	DONE ✓	IMPLEMENTATION	CONFIRM
Receptionist — reservation	6	6	0	0
Receptionist — checked-in questions	10	10	0	0
Receptionist — transfer/split	7	7	0	0
Receptionist — check out	2	2	0	0
Waiter — core	2	2	0	0
Waiter — new order tables	4	4	0	0
Waiter — open orders	6	6	0	0
Waiter — closed orders	4	4	0	0
Waiter — dashboard	2	2	0	0